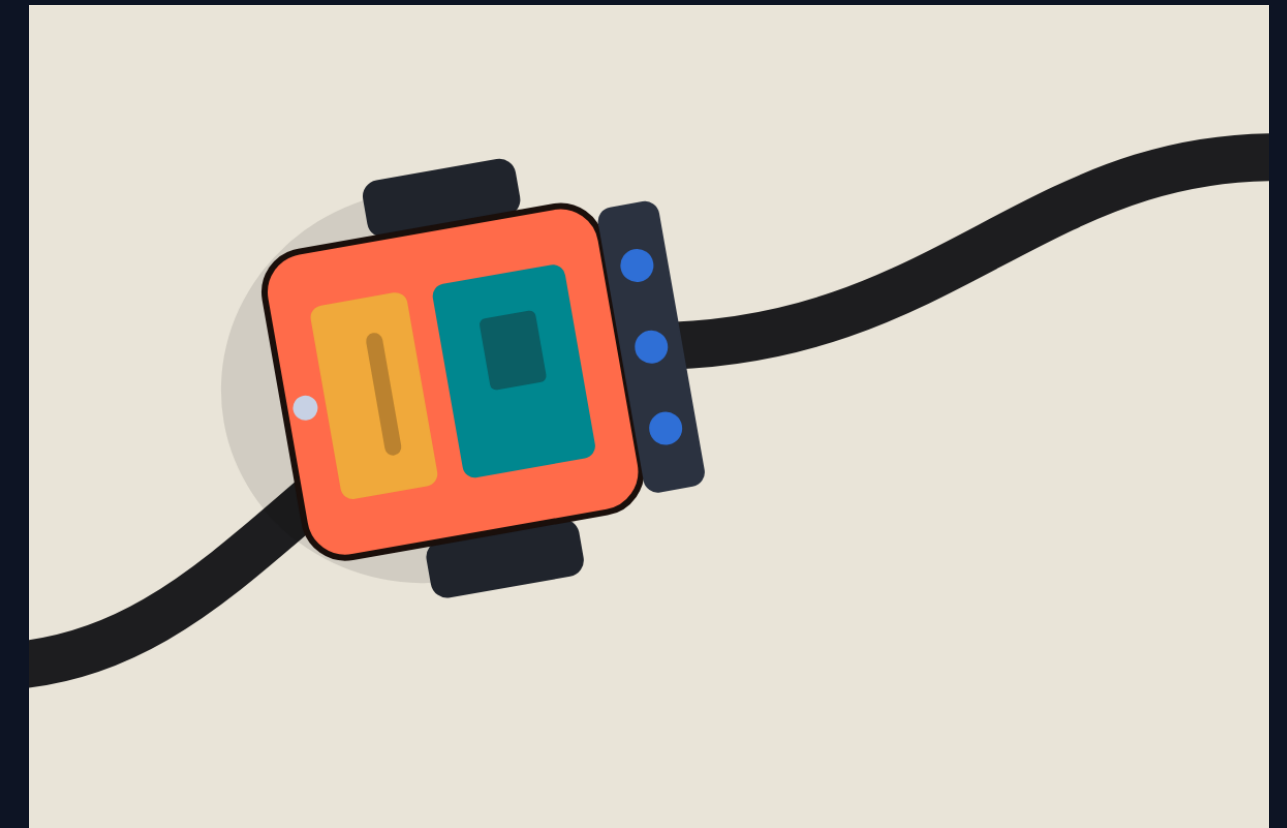


The robot worked yesterday

Today it loses the line. The deadline is approaching.

Arif Solmaz
İSTÜN Mechatronics Engineering



Elif + Deniz · One robot · One shared decision

Which file can we trust?

Elif and Deniz each sent a different “final version”.

THE PROBLEM

Which one is the latest?

🕒 23:47

Robot project


robot.ino


history: every change, who, when

Robot team

K

Kaan

Elif, can you send the latest code?

E

Elif

Sent robot_final.ino

 robot_final.ino

D

Deniz

But I fixed the sensor in robot_v2...

K

Kaan

The robot left the line again. Which one is the latest??

E

Elif

No idea. Should we try them one by one?

On GitHub there is one `robot.ino` ; its old versions live in its history.

Calling a file “FINAL” does not record its history.

We have files. Where is the decision?

The team needs the change's history and reasoning.

What we have

Code files, several copies, and two people's memories.

What we need to find

What changed? Why? Which version should we continue from?

First goal: compare the records, not people's memories.

01 · FOLLOW THE ROBOT'S TRAIL

How do we compare yesterday and today?

There is a clue in the history: threshold 500 → 900.

2 BENEFIT

A time machine for your project

E Elif's screen

FOR YOU

A time machine for your project

✓ No change is ever lost.

✓ Who changed what, and when: all recorded.

✓ Don't be afraid to break things: every save is a way back.

Commit = a save point with a short note.

github.com/elif/my-robot/commit/7f3e2a1

elif / my-robot Public

<> Code Issues 0 Pull requests 0 Actions

Change sensor threshold

deniz committed yesterday 7f3e2a1

1 file changed

robot.ino +1 -1

3 int TURN_SPEED = 120; // speed in turns

4 - int THRESHOLD = 500; // sensor threshold

4 + int THRESHOLD = 900; // sensor threshold

5

6 void setup() {

Found it: the threshold went from 500 to 900. You can look at the project as it was that day.

We found the change. We have not yet established the right threshold.

The robot's settings in the animation

The GIF values are examples. The real threshold needs measurements.

```
// robot.ino – line-following robot
int SPEED = 200;           // speed on straights
int TURN_SPEED = 120;      // speed in turns
int THRESHOLD = 900;       // sensor threshold
```

// starts a comment. **int** defines an integer variable.

= assigns the initial value. **;** ends the declaration.

SPEED, **TURN_SPEED** and **THRESHOLD** name straight speed, turn speed and sensor threshold settings.

Preparing the two motor pins

This fragment prepares pins. The motor-driving code is not shown.

```
void setup() {  
  pinMode(5, OUTPUT); // left motor  
  pinMode(6, OUTPUT); // right motor  
}
```

setup() runs once at startup or reset. **void** means it returns no value.

The comments identify the left and right motor connections. **pinMode** alone does not start either motor.

pinMode(5, OUTPUT) configures pin 5 as an output; the second line does the same for pin 6.

Which part of this is Git?

History lives on the computer. GitHub makes collaboration visible.

Git • local history

Records changes and shows how we arrived at a version.

GitHub • collaboration

Hosts the repository and brings tasks, proposals, and review together.

Git also works with Git servers other than GitHub.

01 · FOLLOW THE ROBOT'S TRAIL

What should we keep together?

The robot's code alone does not explain the project.

1 MINUTE

Put your project on GitHub

E Elif's screen

github.com/elif/my-robot

elif / my-robot Public

<> Code Issues 0 Pull requests 0 Actions

main Add file <> Code

E elif Add project files 2 Commits

photos Add project files

README.md Initial commit

circuit.png Add project files

project-report.pdf Add project files

robot.ino Add project files

README

my-robot

Line-following robot — class project

FOR YOU

Not just code: everything about your project

✓ Arduino code

✓ Circuit diagram

✓ Project report

✓ Photos

Repository = your project's folder on the internet.

Done: code, circuit diagram, report and photos are now online.

Repository = project files + recorded history

What changed on that line?

We are seeing the old and new values of one variable.

```
robot.ino
```

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

robot.ino names the compared file; it is not a program statement.

- marks the removed line and + the added line. These signs are absent from the actual C++ file.

const int ESIK defines an integer constant. Its old value is 500; its new value is 900.

Is 900 wrong?

Git shows the change. We validate the threshold on the robot.

- History: what changed?
- Validation: how does it behave, under which conditions?
- Decision: what evidence would justify accepting it?

500 and 900 are example values in this fictional scenario.

01 · FOLLOW THE ROBOT'S TRAIL

Who is tracking the problem?

Issue: “Investigate the IR sensor threshold.”

3 BENEFIT

Your team's to-do list

github.com/elif/my-robot/issues

elif / my-robot Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open ✓ 1 Closed Author ▾ · Labels ▾ · Assignee ▾

Take robot photos for the report report #3 opened 2 minutes ago by elif

Motor driver gets hot bug #1 opened 2 days ago by kaan

FOR YOU

Everyone sees who's doing what

- ✓ Tasks don't vanish in the group chat; you can find them months later.
- ✓ Every task has an owner.
- ✓ Capstone project, club, assignment group: it fits them all.

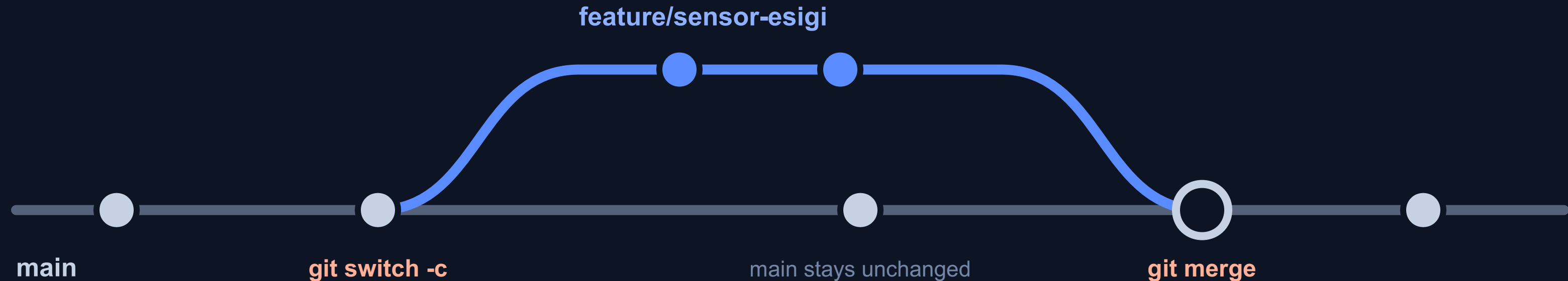
Issue = a task to do or a note about a bug.

When it's done, **Close issue** : it leaves the list, the record stays.

`is:issue` selects issue records; `state:open` selects open ones. Type this in GitHub's search box, not a terminal.

How do we try it without changing main?

We will develop the candidate solution on a separate branch.



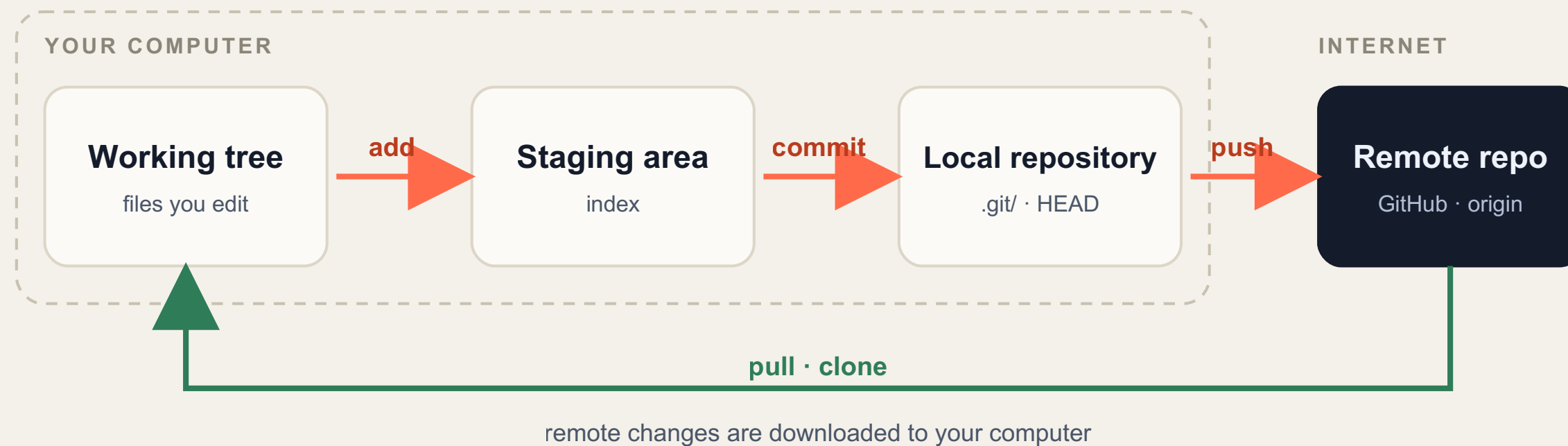
main is the team's shared main branch.

feature/sensor-esigi names the sensor task branch, not a new project folder.

git switch -c creates and switches to a branch. **git merge** combines another branch into the current one.

The diagram uses shortened command labels: **git switch -c** and **git merge**.

I saved it. Can Deniz see it?



index is staging; **.git/** holds local repository data; **HEAD** names the current commit position.

origin is the remote address name. **add** selects, **commit** records locally, and **push** sends commits.

clone creates a local repository copy; **pull** gets updates for an existing one.

How do we get the same starting point?

Elif brings the project's files and history onto her computer.

```
git clone <repository-url>  
cd robot-projem
```

git clone copies an existing repository, including its recorded history, to the computer.

cd robot-projem enters the project folder. Use its actual name; **cd** is a terminal command.

Replace **<repository-url>** with the real clone address, without the angle brackets.

Which branch is the trial on?

Elif creates a separate line for the sensor task.

```
git switch -c feature/sensor-esigi  
git branch
```

git switch -c creates a branch from the current commit and switches to it.

git branch lists local branches. A star marks the active branch.

feature/sensor-esigi is our task branch name. The slash does not create a new project folder.

Which solution are we trying?

We prepare 900 → 500 as a candidate change.

```
robot.ino
```

```
- const int ESIK = 900;  
+ const int ESIK = 500;
```

The removed **900** line is the previous value; the added **500** line is our candidate.

git status checks saved file changes. Saving alone creates no commit and updates no GitHub repository.

Keep only **const int ESIK = 500;** in the real file, without the diff sign.

What goes into this commit?

We select the sensor change. Other work can wait.

```
git add robot.ino  
git diff --cached
```

git add robot.ino selects the file's current content for the next commit.

If the file changes after **add**, its new content must be selected again. Other staged files remain selected.

git diff --cached shows all staged changes against the latest commit.

How do we record what we did?

A commit keeps the change and its purpose in history.

```
git commit -m "Candidate return to 500 for IR sensor threshold"
git log --oneline
```

git commit records the staged content in local history.

git log --oneline lists reachable commits with short IDs and messages. No upload has happened yet.

-m supplies the message. Quotes keep the words together as one message.

Can Deniz see it now?

After push, the candidate is visible on GitHub's task branch.

```
git push -u origin feature/sensor-esigi
```

git push sends local commits to a remote repository.

-u sets that branch's tracking relationship for later commands.

origin is the saved name of the remote address;
feature/sensor-esigi is the branch being sent.

This needs access and write permission. Uncommitted edits are not sent; **main** is not merged.

Should we bring this into main?

A pull request proposes a change for the team to evaluate.

```
feature/sensor-esigi → main
```

Problem: The robot loses the line.

Candidate: ESIK, 900 → 500.

Reason: Investigate a return to the old value.

Validation: Measurement and behavior records pending.

feature/sensor-esigi is the source branch; **main** is the proposed destination.

Validation says what is still pending. This is PR description text, not a terminal command.

Problem, Candidate and Reason explain the need, the change and its purpose.

What will Deniz look for?

The changed line needs a clear reason and expected behavior.

- Why this value?
- Under which conditions will it be validated?
- Does it affect another behavior?

Feedback → new commit on the same branch → updated PR

What if two decisions change one line?

Elif's candidate is 500. Incoming main has 700.

```
robot.ino

<<<<<<< HEAD
const int ESIK = 500;
=====
const int ESIK = 700;
>>>>>>> main
```

<<<<<<< HEAD opens our current branch's version, with value 500.

>>>>>>> main closes the incoming side. These are Git markers, not C++ statements.

===== separates the alternatives. The incoming **main** version has value 700.

The team must choose the justified result and remove the markers. **HEAD** does not mean the correct answer.

Which one do we choose?

In this example, Elif and Deniz continue with the 500 candidate.

```
robot.ino  
  
const int ESIK = 500;
```

const defines a constant; **int** is an integer type; **ESIK** is its name.

One definition remains after the conflict markers are removed. Validate it before recording the merge resolution.

= 500 supplies the example value; **;** ends the definition.

How does the decision enter shared history?

With sufficient review and validation, the proposal can enter main.

Before acceptance

A candidate on a task branch, open to discussion and changes.

After acceptance

A decision in shared main, with reasoning available in the project.

Make it visible → review it → accept it

GitHub is current. Is my computer?

We update local main separately.

```
git status  
git switch main  
git pull --ff-only
```

git status reports working-tree and staging changes before we switch branches.

git pull --ff-only gets remote updates and advances the active branch only if its history has not diverged. Otherwise it stops.

git switch main makes local **main** the active branch.

Where does the next teammate start?

We leave running instructions alongside the code.

```
# Line-Following Robot

## Hardware
Arduino, IR sensors, motor driver.

## Getting started
Follow the wiring diagram.
Validate the threshold in your own
conditions.
Record your observations and reasons for
changes.
```

creates a Markdown title; ## creates a section heading.

The other lines are ordinary README text explaining hardware and how to start.

GitHub displays this file as a readable project page. These lines are not terminal commands.

Can everyone see it and change it?

Visibility and permission to modify are separate decisions.

Viewing

Public: anyone can view. Private: access is limited by permission.

Modifying

Writing to the main repository requires separate permission.

Passwords and access keys do not belong in project files.

Should we use the code we found?

An example needs compatible hardware and operating conditions.

- Does it match our hardware and environment?
- Are there explanations, examples, and known issues?
- What does the license allow?

Stars do not measure correctness on our robot.

Can we use the code we can see?

The license explains the conditions for sharing and use.

- Read the usage conditions.
- Keep required notices.
- State your own sharing conditions clearly.

Public visibility does not grant unrestricted usage rights.

How do we propose a fix to another project?

Push to our fork, then propose a PR to the original project.

1. Read the contribution guide.
2. Fork → clone → task branch → commit → push.
3. Propose a PR from your branch to the original project.

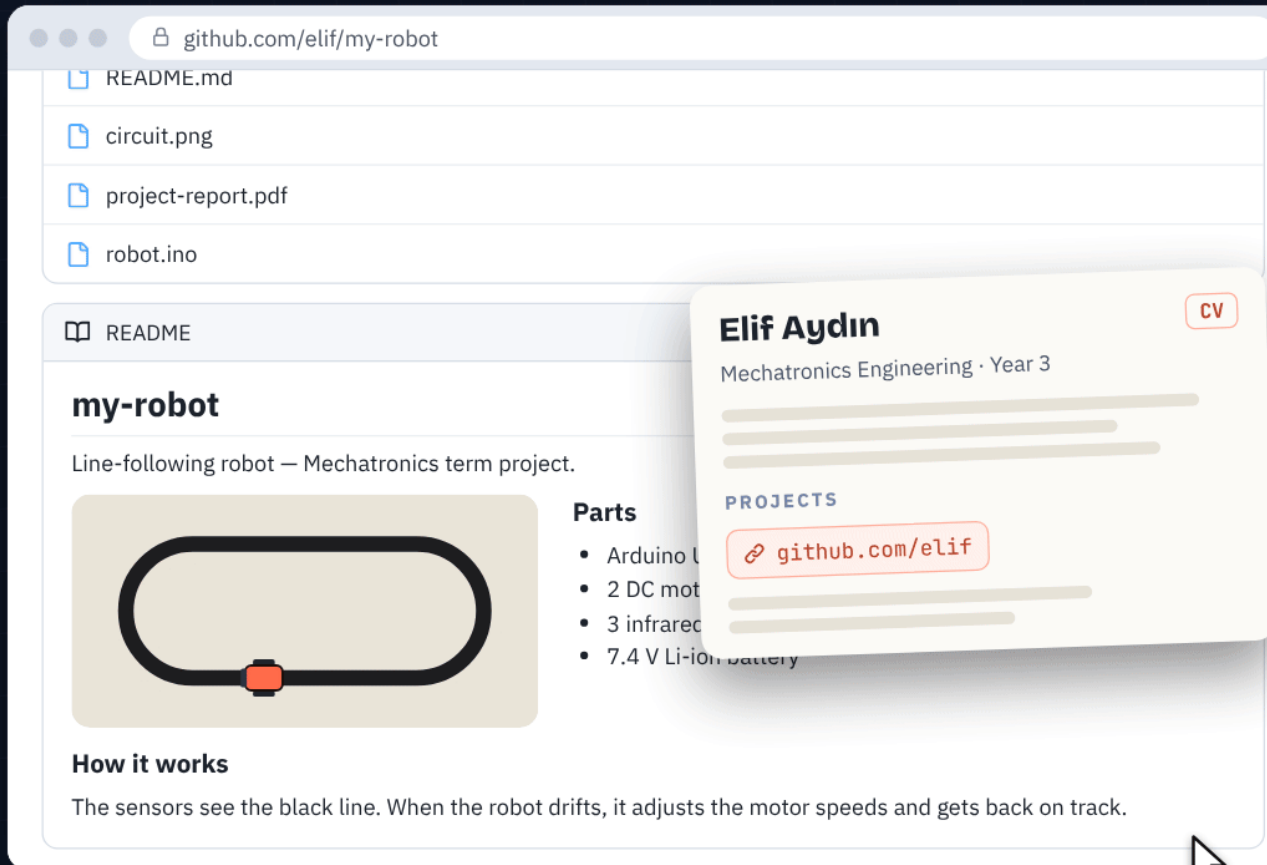
Fork: a remote copy. Clone: a copy on the computer.

04 • HAND OVER THE PROJECT

What does this project say about us?

Make the work behind the files understandable.

5 BENEFIT Your GitHub profile is your portfolio



The screenshot shows a GitHub repository named 'my-robot' by user 'elif'. The repository contains files: README.md, circuit.png, project-report.pdf, and robot.ino. The README.md file is open, showing a title 'my-robot', a description 'Line-following robot — Mechatronics term project.', a diagram of a line-following robot on a track, a list of parts (Arduino Uno, 2 DC motors, 3 infrared sensors, 7.4 V Li-ion battery), and a section 'How it works' explaining the robot's operation. A CV overlay is positioned over the README, showing the user's name 'Elif Aydın', their degree 'Mechatronics Engineering · Year 3', and a link to their GitHub profile 'github.com/elif'. A mouse cursor points to the CV link in the README.

FOR YOU

Your projects = your portfolio

- ✓ Teachers and employers see what you built, not just read about it.
- ✓ Every project is a showcase: photos, diagrams and notes in the README.
- ✓ Add the link to your CV for internships and jobs.

README = your project's front page: what, why, how.

Applying for an internship or a job? One line on your CV: github.com/elif

Problem · your contribution · reasoning · demonstration · known limits

Where can we continue after class?

Return to official resources when a need arises.

- Concepts: Pro Git
- Operation details: GitHub Docs
- Practice at your own pace: GitHub Skills

Setup and commands: 15 reference slides • Student tools: GitHub Education

What does the team have now?

We can explain which change we shared and why.

At the start

Different “final” files. Lost reasoning. Unclear local versions.

Now

Threshold reasoning in the PR. Current main on the team’s computers. A README ready for handover.

Git records the change. The team decides with a reason.

What if it loses the line tomorrow?

Our first question is clear: “What changed, and what is the evidence?”

Where is the change? Who can see it? Why accept it?

[Git and GitHub learning resources](#)

[Pro Git · English](#)

[GitHub Skills](#)

[GitHub Education](#)

Installing Git

Use the official installation route for your operating system.

```
git --version
```

git --version prints the Git version available to the terminal.

It checks the installation; it does not install Git or create a repository.

Commit identity

Configure the name and email that will appear in commits.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

git config --global sets values for this user account. A repository can override them locally.

init.defaultBranch main sets the first branch name for future repositories; it renames no existing branch.

user.name and **user.email** label new commits. They do not sign you in to GitHub.

Editor and terminal

The same Git operation can be done in a terminal or a graphical interface.

Terminal

Commands and options are explicit, making steps easy to share.

VS Code

Files, diffs, Source Control, and an integrated terminal in one window.

Open the project with Open Folder; show the terminal with View → Terminal.

.git and .gitignore

The history database and tracking preferences are stored separately.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# Example .gitignore  
build/  
.env
```

robot-projem/ is the project folder. **.git/** holds local Git history and settings.

README.md explains the project; **robot.ino** contains the source code.

In **.gitignore**, **build/** excludes that untracked directory and **.env** excludes that untracked filename.

starts a comment. Ignore rules erase neither tracked content nor past secrets.

An SSH key pair

Add the public key to GitHub; keep the private key to yourself.

```
ssh-keygen -t ed25519 -C "elif@example.com"

# File to add to GitHub:
~/.ssh/id_ed25519.pub

# Private key: do not share
~/.ssh/id_ed25519
```

ssh-keygen creates a key pair. **-t ed25519** selects its type; **-C** adds a descriptive label.

Add the **.pub** public key to GitHub. Keep the private key without **.pub** to yourself.

~ means the user's home directory. The example paths are filenames, not extra commands.

SSH agent and connection

Agent setup depends on the operating system.

```
# Bash / Git Bash example
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Check the GitHub connection
ssh -T git@github.com
```

begins a comment. This setup example uses Bash or Git Bash.

ssh-add loads the private key into the agent.

GitHub offers no shell access. Successful **ssh -T** authentication normally returns exit code 1.

ssh-agent -s starts an agent; **\$(...)** captures its settings; **eval** applies them to this shell.

ssh -T checks GitHub authentication without requesting a terminal.

Starting a local repository

The folder, README.md and robot.ino already exist.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino
```

cd robot-projem enters an existing folder; it creates no folder.

git add README.md robot.ino stages these two files. Nothing has been committed or uploaded yet.

git init -b main starts a local repository with **main** as its first branch name.

First commit and remote address

The GitHub repository has already been created and is empty.

```
git commit -m "Initial robot project commit"  
git remote add origin <repository-url>  
git remote -v
```

git commit -m records the staged content with the quoted message in local history.

Replace **<repository-url>** without angle brackets. **git remote -v** lists fetch and push addresses; it uploads no files.

git remote add origin saves the real remote address under the name **origin**.

First push and tracking

-u records which remote branch the local branch tracks.

```
git push -u origin main

# After setting up tracking:
git push
git pull --ff-only
```

git push -u origin main sends local **main** commits and records the tracking relationship.

git pull --ff-only gets updates and stops if histories diverge. These examples are not a mandatory sequence.

With suitable tracking and configuration, **git push** sends later commits without repeating the remote and branch.

Task branch and current branch

This reference example assumes the new branch does not already exist.

```
git branch  
git switch -c feature/sensor-esigi
```

git branch lists local branches and marks the active one with a star.

Here **feature/sensor-esigi** is the branch name. To start from **main**, first be on **main**.

git switch -c creates the named branch from the current commit and switches to it.

Two ways to compare the branches

The two comparisons use different starting points.

```
# Compare branch tips: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Feature changes from the common starting point:
git diff main...feature/sensor-esigi
```

git diff main feature/sensor-esigi compares the final states of both branches.

main...feature/sensor-esigi compares the common starting commit with the task branch tip.

-- robot.ino limits comparison to this file; **--** separates branch names from the path.

Choose what you want to undo

HEAD~1: previous commit · <commit>: chosen commit ID

Unstage a change

```
git restore --staged README.md
```

Unstages the file. Working-file edits remain.

Reverse a shared change

```
git revert <commit>
```

Creates a new commit that reverses the selected commit's effect.

`git reset HEAD~1` moves the branch back; default `mixed` mode resets staging to the target and keeps working files.

Hard reset and recovery limits

A hard reset can discard uncommitted work.

```
# Only after checking the effects:  
git reset --hard <commit>  
  
# Inspect local reference movements:  
git reflog
```

git reset --hard <commit> moves the branch and replaces staging and tracked working files with that commit's state.

git reflog lists local reference movements. It may locate earlier commits but cannot recover every unsaved change.

Uncommitted edits can be lost. Some untracked files in the way can also be affected.

Selecting committing and sending a file

Choose the next operation from the state of the actual project.

```
git clone <repository-url>
# Edit and save the file
git status
git add <file>
git commit -m "What changed and why?"
git push
```

git clone copies a repository. Then enter its project folder.

git status checks files and staging.

git commit -m records staging with a message; **git push** sends commits.

starts a comment reminding us to edit and save first.

git add <file> selects that file's current content.

Replace **<repository-url>** and **<file>** with real values.

Remote updates and a new task branch

Use pull for remote updates; use a branch for a separate task.

```
# Get the team's updates
git pull --ff-only
# Create a task branch
git switch -c feature/<task>
```

git pull --ff-only fetches remote changes and advances the active branch only if histories have not diverged.

introduces explanatory comments. Pull needs suitable remote tracking; branch creation does not require a new folder.

git switch -c feature/<task> creates and switches to a task branch. Replace **<task>** with its actual short name.